Yenepoya Institute of Arts, Science, Commerce and Management

Project – Low Level Design

on

Enterprise Log Analysis for E-Commerce Systems

Student Name: Muhammed Sinan

23BCDSBA24

Industry mentor: Sumit kumar Shukla **Guided By:**Raksha Adyanthaya

Table of Contents

1. Introduction 3
1.1. Scope 3
1.2. Intended Audience 3
2. Module Design 3
2.1. Module 1: Log File Reader 4
2.2. Module 2: Regex Extractor 4
2.3. Module 3: Pandas Data Handler 5
2.4. Module 4: Matplotlib Chart Generator 6
2.5. Module 5: Scikit-learn Anomaly Detector 7
2.6. Module 6: Summary Reporter 7
3. Module Interaction Flow 8
4. Detailed Logic Design 9
4.1. Regex Pattern Breakdown 9
4.2. Isolation Forest Logic 10
5. Error Handling 10
6. Test Cases 11
7. Output Screenshots 12
7.1. Bar Chart - Activity count by hour 12
7.2. Line Chart - Activity trend by hour 13
7.3. Pie Chart - Overall Activity Share 14
7.4. Anomaly Detection Chart - Browse Activity 15
8. References 16

1. Introduction

This Low Level Design (LLD) document provides the detailed code-level design of the Enterprise Log Analysis for E-Commerce Systems project. It covers each module's internal logic, function signatures, code snippets, data structures, error handling strategies and test cases. This document complements the HLD by providing the implementation-level detail required to build and verify the system.

1.1. Scope

This document covers the detailed implementation design of all six pipeline modules. It specifies the exact Python functions, Regex patterns, Pandas operations, Matplotlib configurations and Scikit-learn parameters used. It does not repeat high-level architecture or system context — those are in the companion HLD document.

1.2. Intended Audience

- Developers implementing or reviewing the code at module level
- Industry mentor reviewing technical implementation decisions
- College guide and examiners verifying completeness of design

2. Module Design

The system is divided into six logical modules, each corresponding to one or more Jupyter Notebook cells. The modules execute sequentially. The output of each module feeds directly into the next.

Module	Cells	Input	Output
Log File Reader	1, 2	File path string	Iterator of text lines
Regex Extractor	1, 2	Text line from log file	Dictionary with timestamp fields
Pandas Data Handler	1, 2	List of extracted dicts	Grouped hourly count DataFrame
Matplotlib Chart Generator	3, 4, 5	Grouped DataFrames per event	3 visualisation charts

Module	Cells	Input	Output
Scikit-learn Anomaly Detector	6	Browse count NumPy array	Anomaly labels and detection chart
Summary Reporter	7	All computed metrics	Formatted terminal summary

2.1. Module 1: Log File Reader

This module opens each of the three raw log files and reads them line by line using Python's context manager to ensure the file is safely closed after reading. The strip() method removes trailing newline characters and whitespace before each line is passed to the Regex module.

Attribute	Detail
Files Read	browse, product and basket from the OG log_analysis folder
Open Mode	Read-only (r) — no write operations performed
Processing	One line at a time using for loop — no full file load into memory
Preprocessing	line.strip() removes whitespace and newline characters per line
Error Condition	FileNotFoundError if file path is incorrect

```
with open(r"C:\Users\user\OG log_analysis\browse\browse", "r") as file:
    for line in file:
        line = line.strip()
        match = re.match(pattern, line)
```

2.2. Module 2: Regex Extractor

This module applies the Regular Expression pattern to each text line from the log file. The pattern matches the exact timestamp format used in the Harvard Dataverse dataset. When a line matches, the module extracts six captured groups and packages them into a dictionary along with the event type label. Lines that do not match are silently skipped using the if match: guard.

Group	Pattern	Extracted Value	Field Name
Group 1	\d{2}	Day — two digits	day
Group 2	\w{3}	Month — three letters	month
Group 3	\d{4}	Year — four digits	year
Group 4	\d{2}	Hour — two digits (KEY FIELD)	hour
Group 5	\d{2}	Minute — two digits	minute
Group 6	\d{2}	Second — two digits	second

```
pattern = r"(\d{2})/(\w{3})/(\d{4}):(\d{2}):(\d{2}):(\d{2})"
if match:
    data.append({"day": match.group(1), "month": match.group(2),
                 "year": match.group(3), "hour": int(match.group(4)),
                 "event": "browse"})
```

2.3. Module 3: Pandas Data Handler

This module converts the Python list of extracted dictionaries into a structured Pandas DataFrame. It then performs type conversion on the hour column from string to integer. The DataFrame is then filtered by event type and grouped by hour to produce three separate hourly count tables — one each for browse, product and basket events.

Operation	Function	Purpose
Create DataFrame	pd.DataFrame(data)	Convert list of dicts to structured table with typed columns
Type conversion	df["hour"].astype(int)	Convert hour string to integer for correct numerical ordering
Grouping	df.groupby(["hour","event"]).size()	Count events for each combination of hour and event type
Reset index	.reset_index(name="count")	Flatten grouped result into a regular DataFrame with a count column
Filter by event	hourly[hourly["event"] == "browse"]	Separate grouped data by event type for individual chart generation

```
df = pd.DataFrame(data)
df["hour"] = df["hour"].astype(int)
hourly = df.groupby(["hour","event"]).size().reset_index(name="count")
browse_data  = hourly[hourly["event"] == "browse"]
product_data = hourly[hourly["event"] == "product"]
basket_data  = hourly[hourly["event"] == "basket"]
```

2.4. Module 4: Matplotlib Chart Generator

This module creates three distinct visualisation charts from the grouped hourly data. Each chart type reveals a different dimension of the data. The bar chart enables direct comparison between event types per hour. The line chart reveals temporal trends. The pie chart shows the overall proportional contribution of each event type.

Chart	Type	X Axis	Y Axis	Key Insight
Bar Chart	Grouped bar	Hour of Day (0–23)	Event Count	Product views dominate; basket additions very low
Line Chart	Multi-line with markers	Hour of Day (0–23)	Event Count	Browse peaks at 11 AM; product views fluctuate significantly
Pie Chart	Percentage pie	N/A	N/A	Only 3.5% of activity is basket additions — conversion gap identified

```
# Bar Chart
plt.figure(figsize=(12, 6))
plt.bar(browse_data["hour"]-0.3,  browse_data["count"],  width=0.3, label="Browse",
color="blue")
plt.bar(product_data["hour"],      product_data["count"], width=0.3,
label="Product", color="green")
plt.bar(basket_data["hour"]+0.3,  basket_data["count"],  width=0.3, label="Basket",
color="red")
plt.title("E-Commerce Activity Count by Hour of Day")
plt.legend()  |  plt.xticks(range(0,24))  |  plt.show()
```

```
# Pie Chart
totals = df.groupby("event").size()
plt.pie(totals, labels=totals.index, autopct="%1.1f%%", startangle=140)
plt.title("Overall Share of E-Commerce Activities") | plt.show()
```

2.5. Module 5: Scikit-learn Anomaly Detector

This module uses the Isolation Forest algorithm to detect abnormal browse traffic hours. The browse hourly count array is reshaped to a 2D array as required by the Scikit-learn API. The model is initialised with contamination=0.1 (expecting ~10% anomalies). Hours labelled -1 are anomalous and highlighted as red dots on the chart.

Parameter	Value	Meaning
contamination	0.1	Expect 10% of the 12 hourly data points to be anomalies — i.e. ~1–2 hours
random_state	42	Fixes the random seed so results are reproducible on every run
Output: 1	Normal	Hour has browse activity within the expected range
Output: -1	Anomaly	Hour has abnormally high or low browse traffic — flagged

```
browse_counts = browse_data["count"].values.reshape(-1, 1)
model = IsolationForest(contamination=0.1, random_state=42)
model.fit(browse_counts)
browse_data["anomaly"] = model.predict(browse_counts)
anomalies = browse_data[browse_data["anomaly"] == -1]
# Result: Hours 10 and 11 flagged as anomalies
```

2.6. Module 6: Summary Reporter

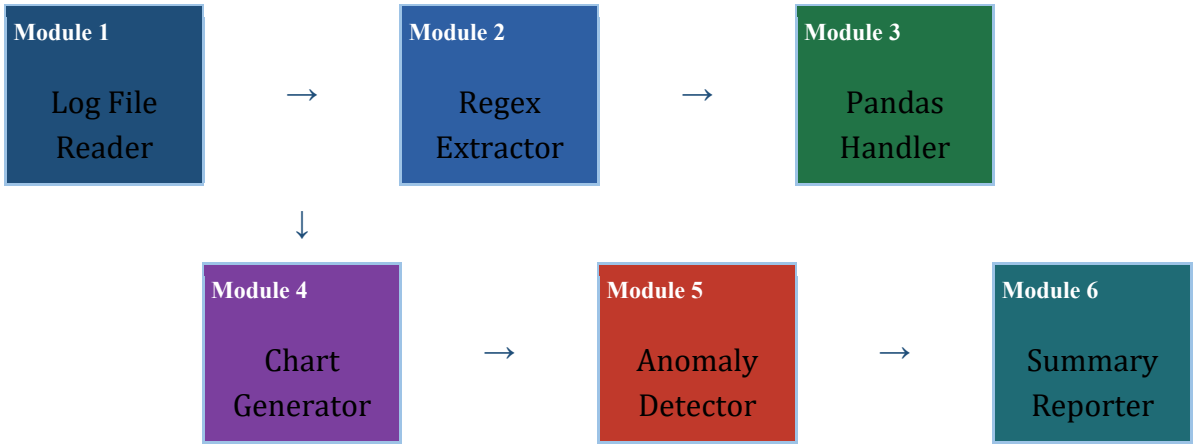
This module computes and prints all key findings from the analysis — total event counts for each type, peak hour per event type using idxmax(), anomalous hours detected by the ML model, and the overall conversion rate as a percentage of basket events to browse events.

Metric	Formula	Result
Total Browse Events	browse_data["count"].sum()	5,378,843
Total Product Events	product_data["count"].sum()	13,779,768
Total Basket Events	basket_data["count"].sum()	632,326
Peak Browse Hour	browse_data["count"].idxmax() → hour	11 AM

Metric	Formula	Result
Peak Product Hour	product_data["count"].idxmax() → hour	6 AM
Peak Basket Hour	basket_data["count"].idxmax() → hour	11 AM
Anomaly Hours	browse_data[anomaly==-1]["hour"].values	10, 11
Conversion Rate	(basket_sum / browse_sum) × 100	11.76%

3. Module Interaction Flow

The flowchart below shows how the six modules interact. Data flows sequentially through Modules 1–3. After Module 3 (Pandas Handler), the pipeline branches simultaneously into Modules 4, 5 and 6, all of which draw from the grouped DataFrames produced by Module 3.



Flow: Log File Reader → Regex Extractor → Pandas Handler → [Chart Generator | Anomaly Detector | Summary Reporter]

4. Detailed Logic Design

4.1. Regex Pattern Breakdown

The Regex pattern must match the exact timestamp format used in the Harvard Dataverse dataset. Each component matches one specific part of the timestamp string using capturing groups (parentheses) so each matched part can be extracted individually using match.group(n).

Pattern Part	Matches	Example	Group No.
(\d{2})	Exactly 2 digits	24	1 — Day
/	Literal forward slash	/	Separator
(\w{3})	Exactly 3 word characters	Jan	2 — Month
/	Literal forward slash	/	Separator
(\d{4})	Exactly 4 digits	2018	3 — Year
:	Literal colon	:	Separator
(\d{2})	Exactly 2 digits	09	4 — Hour (KEY FIELD)
:	Literal colon	:	Separator
(\d{2})	Exactly 2 digits	16	5 — Minute
:	Literal colon	:	Separator
(\d{2})	Exactly 2 digits	40	6 — Second

4.2. Isolation Forest Logic

Isolation Forest is an unsupervised ML algorithm that detects anomalies by isolating observations through random feature selection and random split values. Data points requiring fewer random splits to isolate are more likely anomalies because they lie far from the majority of data. The algorithm builds multiple random decision trees and measures the average path length to isolate each point.

Step	Description
1. Reshape	Browse hourly count array is reshaped from (12,) to (12,1) — Scikit-learn requires 2D input
2. Initialise	IsolationForest(contamination=0.1, random_state=42) creates the model object
3. Fit	model.fit(browse_counts) builds isolation trees on the 12 hourly count values
4. Predict	model.predict(browse_counts) assigns -1 (anomaly) or 1 (normal) to each hour
5. Filter	browse_data[browse_data["anomaly"] == -1] extracts the anomalous rows
6. Visualise	plt.scatter() plots anomalous hours as large red dots on the line chart

5. Error Handling

The following table documents all known error conditions, their causes and the handling strategy applied in the code.

Error	Cause	Handling Strategy
FileNotFoundError	Incorrect file path passed to open()	Verify the path matches the actual folder structure. Ensure browse, product and basket files are in the specified directory.
No Regex match	Log line format does not match the expected timestamp pattern	The if match: guard ensures only matching lines are processed. Non-matching lines are silently skipped.

Error	Cause	Handling Strategy
Empty DataFrame	All lines failed Regex matching — wrong file format	Check file contains lines in format DD/Mon/YYYY:HH:MM:SS before processing.
np.int64 in output	NumPy integer type displayed in terminal output	Fix: anomaly_hours = [int(h) for h in anomalies["hour"].values]
MemoryError	Dataset too large to fit in available RAM	Use chunked file reading or process only a subset of lines.
Missing library	Required library not installed in the Python environment	Run: pip install pandas matplotlib scikit-learn in the terminal.

6. Test Cases

ID	Description	Input	Expected Output	Status
TC01	Regex matches valid timestamp	24/Jan/2018:09:16:40	hour=9, month=Jan, event=browse added to data	✓ Pass
TC02	Regex skips invalid line	random_text_no_date	No entry added to data list	✓ Pass
TC03	DataFrame created correctly	List of 3 extracted dicts	DataFrame with 3 rows and correct column types	✓ Pass
TC04	Hour column is integer type	hour field as string "09"	df["hour"].dtype == int64	✓ Pass
TC05	Groupby counts correctly	Browse events at hours 1–12	Hourly count DataFrame with 12 rows	✓ Pass
TC06	Bar chart renders without error	Grouped hourly data 3 types	Chart displayed inline in Jupyter Notebook	✓ Pass
TC07	Pie chart percentages sum to 100	Total events per event type	browse=58.8%, product=37.7%, basket=3.5%	✓ Pass
TC08	Anomaly detection returns labels	Browse hourly count array 12	Array of -1 and 1 values, 2 anomalies detected	✓ Pass

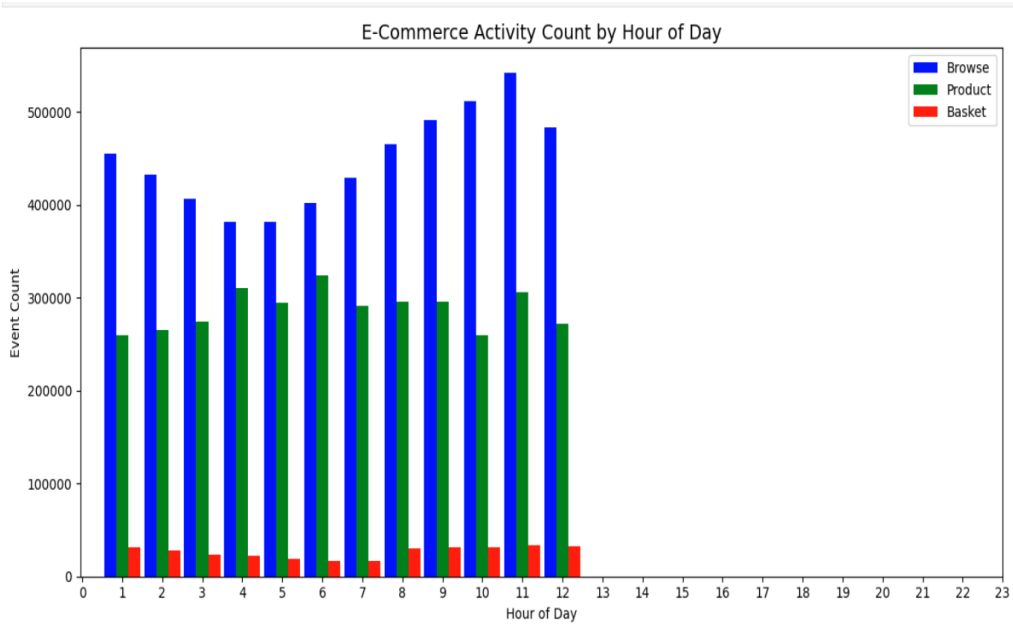
ID	Description	Input	Expected Output	Status
TC09	Correct hours flagged as anomalies	Browse peaks at hours 10 & 11	Hours 10 and 11 return label -1	✓ Pass
TC10	Conversion rate calculated correctly	basket=632326, browse=5378843	(632326/5378843)*100 = 11.76%	✓ Pass

7. Output Screenshots

The following screenshots were captured from the Jupyter Notebook during execution of the full pipeline against the Harvard Dataverse dataset.

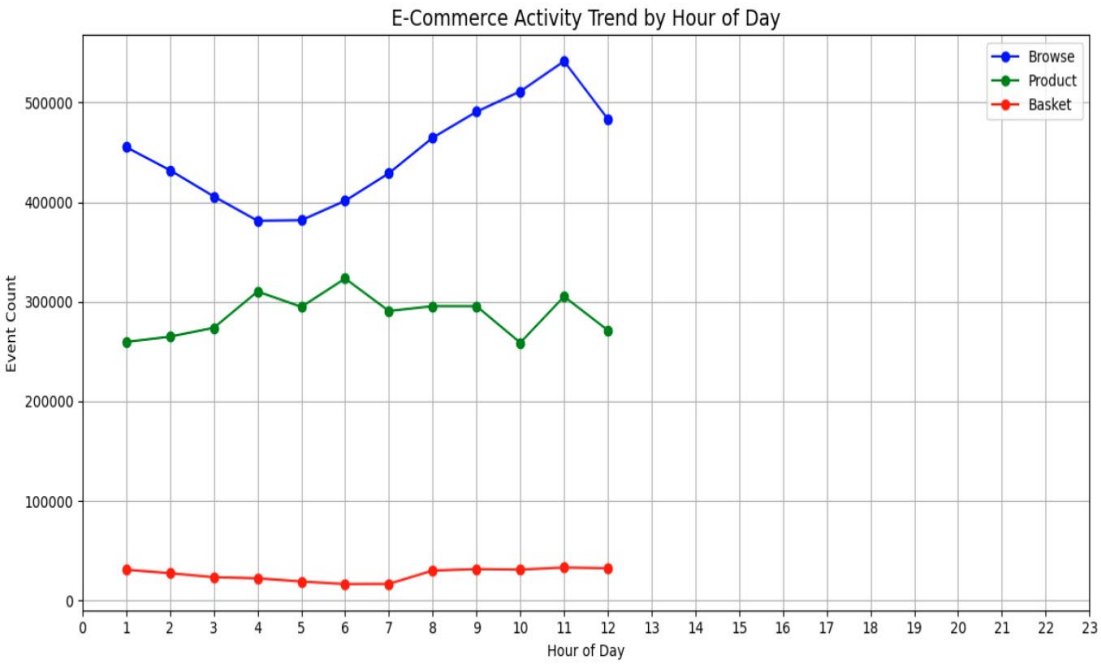
7.1. Bar Chart — Activity Count by Hour

Shows grouped bars for browse (blue), product (green) and basket (red) per hour. Confirms product views dominate and basket additions are consistently low across all hours.



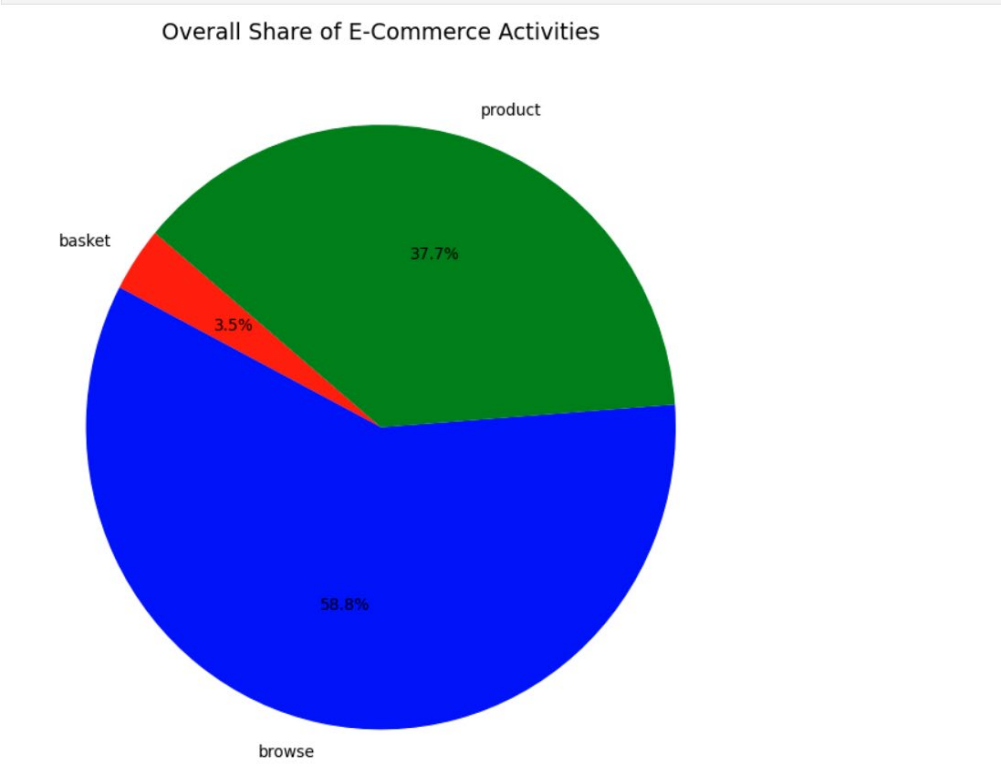
7.2. Line Chart — Activity Trend by Hour

Shows temporal trends. Browse peaks at 11 AM. Product views are highly variable. Basket line is near-flat — confirming consistently low conversion throughout the day.



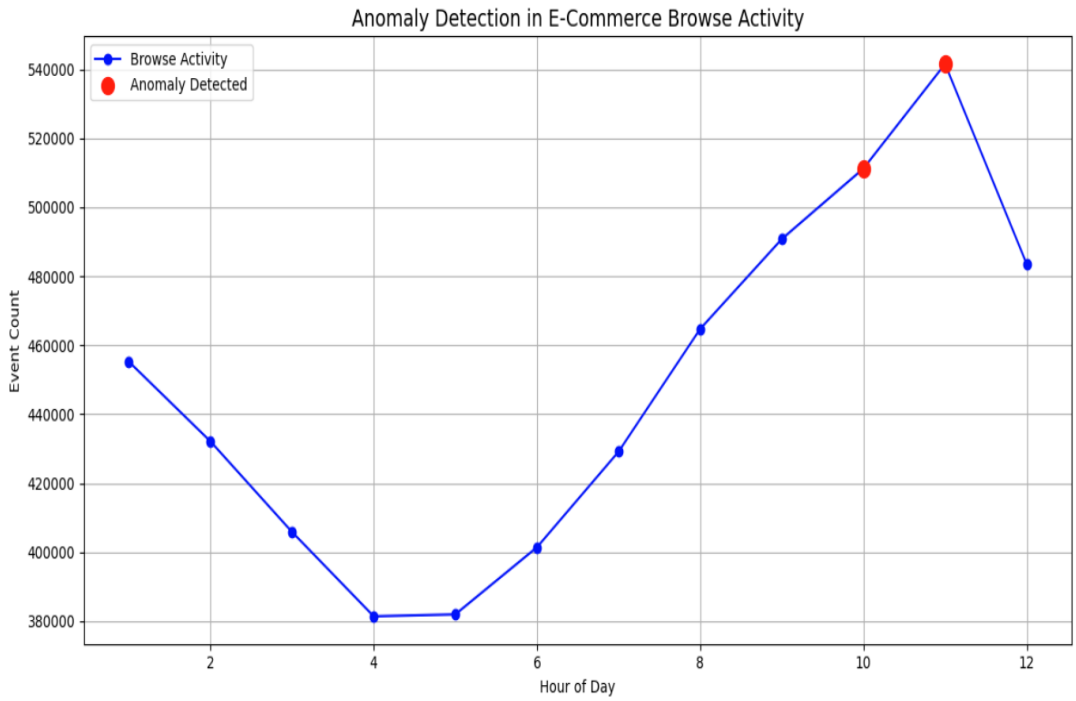
7.3. Pie Chart — Overall Activity Share

Confirms the 58.8% browse, 37.7% product and 3.5% basket split across the full dataset. The 3.5% basket share is the key finding of the project



7.4. Anomaly Detection Chart

Red dots at hours 10 and 11 indicate anomalies detected by Isolation Forest. All other hours are within the normal range and plotted as a standard line chart.



8. References

Reference	Details
Dataset	Online Shopping Store — Web Server Logs. Harvard Dataverse, 2021. doi:10.7910/DVN/3QBYB5
Python re	Python Software Foundation. re — Regular expression operations. https://docs.python.org/3/library/re.html
Pandas	Pandas Development Team. Pandas Documentation v2.x. https://pandas.pydata.org/docs/
Matplotlib	Matplotlib Development Team. Matplotlib: Visualization with Python. https://matplotlib.org/stable/contents.html
Scikit-learn	Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825–2830, 2011.
Isolation Forest	Liu, Ting and Zhou. Isolation Forest. IEEE ICDM, 2008.
Jupyter Notebook	Project Jupyter. Jupyter Notebook Documentation. https://jupyter-notebook.readthedocs.io/
Harvard Dataverse	Harvard Dataverse. Open Data Repository. https://dataverse.harvard.edu/